A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Practical No: 1

Aim: Study of web analytics using open source tools like Matomo, Open Web Analytics, Plausible.

Theory:
Web analytics is the process of collecting, analyzing, and interpreting data related to website usage. It helps organizations understand user behavior, improve website performance, and make data-driven decisions. With the rapid growth of digital platforms, web analytics has become essential for businesses, educational institutions, and developers. Web analytics answers critical questions such as:

1. How many users visit a website?
2. Which pages are most popular?
3. What is the user journey?
4. Where do users drop off?

Traditionally, tools like Google Analytics dominate the market, but concerns about data privacy, ownership, and customization have led to the rise of open-source alternatives.

Open-source web analytics tools allow organizations to host data on their own servers, ensuring better control, transparency, and compliance with privacy regulations like GDPR.

Importance of Open Source Web Analytics Tools

Open-source web analytics tools provide several advantages over proprietary solutions:

1. Data Privacy and Ownership
   Users maintain full control over their data. Sensitive information is not shared with third-party companies.
2. Customization
   Developers can modify the source code to meet specific needs such as custom dashboards, reports, or integrations.
3. Cost-Effective
   These tools are generally free to use, reducing the cost for startups and small organizations.
4. TransparencySince the code is publicly available, users can verify how data is collected and processed.
5. Compliance
   Helps organizations comply with regulations like GDPR and CCPA.

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Overview of Open Source Web Analytics Tools

1. Matomo
   Matomo (formerly Piwik) is one of the most popular open-source web analytics platforms.
   Features:
   i. Real-time data tracking
   ii. Heatmaps and session recordings
   iii. Goal tracking and conversion analysis
   iv. Custom reports and dashboardsGDPR compliance tools
   Working:
   Matomo uses a JavaScript tracking code embedded into web pages. When users visit the site, data such as page views, session duration, and user interactions are recorded and stored in a database.
   Advantages:
   i. Full data ownership
   ii. Highly customizable
   iii. Supports plugins and extensions
   Limitations:
   i. Requires server setup and maintenance
   ii. May need technical knowledge for advanced features
2. Open Web Analytics (OWA)
   Open Web Analytics is another open-source analytics tool that provides detailed tracking of website visitors.
   Features:
   i. Clickstream tracking
   ii.Heatmaps
   iii.Visitor segmentation
   iv.Event trackingAPI support
   Working:
   OWA tracks user activity using JavaScript or PHP-based tracking. It logs visitor data such as clicks, page views, and referrer information into a database for analysis.
   Advantages:
   i.Easy integration with websites
   ii. Supports multiple websites tracking
   iii. Open API for developers
   Limitations:
   i. Less active development compared to Matomo
   ii. Limited advanced features

3. Plausible
   Plausible is a lightweight and privacy-focused open-source analytics tool.
   Features:
   - i. Simple and clean dashboard
   - ii.No cookies required
   - iii.Lightweight script (faster website performance)
   - iv.GDPR, CCPA compliant
   - v.Real-time analytics

   Working:
   Plausible uses a minimal JavaScript script to track page views and basic metrics. It does not use cookies, making it privacy-friendly.
   Advantages:
   - i.Easy to use
   - ii.Privacy-first approach
   - iii.Fast performance

   Limitations:
   - i.Limited advanced analytics features
   - ii.Less detailed reporting compared to Matomo

Working Principle of Web Analytics Tools
The general working of web analytics tools involves the following steps:

1. Data Collection
   A tracking code (JavaScript snippet) is embedded in the website.
2. Data Transmission
   User interactions are sent to the analytics server.
3. Data Storage
   The collected data is stored in databases.
4. Data Processing
   Raw data is processed into meaningful metrics such as sessions, bounce rate, and conversions.
5. Data Visualization
   Results are displayed through dashboards, graphs, and reports.

Applications of Web Analytics

1. Business Optimization: Improve marketing strategies and ROI
2. User Experience Enhancement: Identify usability issues
3. Content Strategy: Analyze which content performs best
4. E-commerce Tracking: Monitor sales and conversions
5. Academic Research: Study user behavior patterns

Challenges in Web Analytics

1. Data privacy concernsHandling large datasets
2. Accurate tracking (ad blockers, cookie restrictions)
3. Technical complexity in setup
4. Data interpretation difficulties

Future Scope

The future of web analytics lies in:

1. AI-driven insights and predictive analytics
2. Increased focus on privacy-first tracking
3. Integration with machine learning models
4. Real-time personalization of content
5. Cookieless tracking technologies

Open-source tools are expected to grow significantly due to increasing awareness of data privacy and the need for transparent analytics solutions.

Conclusion:

Web analytics plays a crucial role in understanding and improving online platforms. Open-source tools like Matomo, Open Web Analytics, and Plausible offer powerful alternatives to proprietary solutions by providing better control, privacy, and customization. While each tool has its strengths and limitations, they collectively empower organizations to make informed decisions based on reliable data. As digital ecosystems evolve, the adoption of open-source analytics tools is likely to increase, making them essential components of modern web development and data analysis.

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Practical No: 2

Aim: To study the basics of TypeScript through small code snippets
A) Write a Typescript program for Hello World.
B) Design a Calculator using TypeScript.

Theory:
TypeScript is a statically typed superset of JavaScript designed to enhance code quality, scalability, and maintainability by introducing features such as type annotations, interfaces, and compile-time error checking.
Some features of typescript include:

1. Type Safety: Variables and functions can be defined with specific data types such as string and number, which helps in preventing unintended operations and improves code reliability.

2. Compilation Process: TypeScript code is compiled into JavaScript using the TypeScript compiler (tsc), as browsers and runtime environments execute only JavaScript.

3. Program Structure: Both programs demonstrate structured coding practices using variable declarations, function definitions, and clear logical flow.

4. Execution Environment: The output is displayed either through the console in a terminal or by interacting with the web page using the Document Object Model (DOM).

5. Modular Logic: Functions are used to separate different operations, making the code reusable and easier to understand and maintain.

6. Error Handling: TypeScript helps in identifying type-related and syntax errors at compile time, reducing the chances of runtime failures.

7. User Interaction: The programs show how TypeScript can handle user input and output, either through console messages or graphical interface elements in a browser.

Implementation:

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

a) Hello World Program Using Typescript

```
TS FirstCodeTypescript.ts ×
C: > Users > Shilpa_J > Downloads > DatasetFromPDF > FirstTypescriptTutorial > TS FirstCodeTypescript.ts > ...
1  let message: string = "Hello, World!";
2  console.log(message);
```

Output:

```
C:\Users\Shilpa_J\Downloads\DatasetFromPDF\FirstTypescriptTutorial>tsc FirstCodeTypescript.ts

C:\Users\Shilpa_J\Downloads\DatasetFromPDF\FirstTypescriptTutorial>node FirstCodeTypescript.ts
Hello, World!

C:\Users\Shilpa_J\Downloads\DatasetFromPDF\FirstTypescriptTutorial>
```

b) Calculator Program Using Typescript

1) calculator.ts

```
let display = document.getElementById("display") as HTMLInputElement;
// Add value to display
function press(value: string): void {
  display.value += value;
}
// Clear display
function clearDisplay(): void {
  display.value = "";
}
// Calculate result
function calculate(): void {
  try {
    display.value = eval(display.value);
  } catch {
    display.value = "Error";
  }
}
// Expose functions to window so HTML can use them
(window as any).press = press;
(window as any).clearDisplay = clearDisplay;
(window as any).calculate = calculate;
```

2) index.html

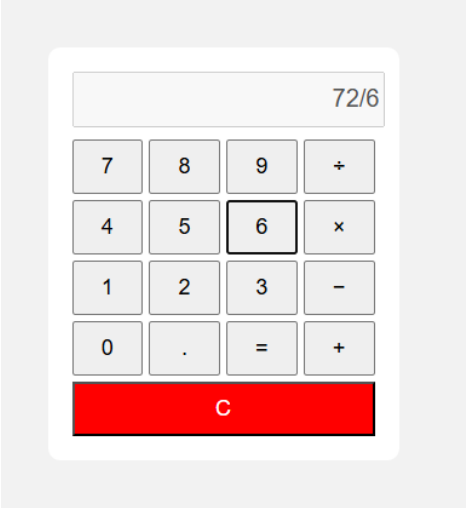
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>TypeScript Calculator</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="calculator">
    <input type="text" id="display" disabled />

    <div class="buttons">
      <button onclick="press('7')">7</button>
      <button onclick="press('8')">8</button>
      <button onclick="press('9')">9</button>
      <button onclick="press('/')">÷</button>

      <button onclick="press('4')">4</button>
      <button onclick="press('5')">5</button>
      <button onclick="press('6')">6</button>
      <button onclick="press('*')">×</button>

      <button onclick="press('1')">1</button>
      <button onclick="press('2')">2</button>
      <button onclick="press('3')">3</button>
      <button onclick="press('-')">−</button>

      <button onclick="press('0')">0</button>
      <button onclick="press('.')">.</button>
      <button onclick="calculate()">=</button>
      <button onclick="press('+')">+</button>

      <button class="clear" onclick="clearDisplay()">C</button>
    </div>
  </div>

  <script src="calculator.js"></script>
</body>
</html>
```

3) style.css

```
body {
```

```
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  background: #f2f2f2;
}

.calculator {
  background: white;
  padding: 20px;
  border-radius: 10px;
  width: 250px;
}

#display {
  width: 100%;
  height: 40px;
  margin-bottom: 10px;
  font-size: 20px;
  text-align: right;
}

.buttons {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 5px;
}

button {
  padding: 10px;
  font-size: 18px;
  cursor: pointer;
}

.clear {
  grid-column: span 4;
  background: red;
  color: white;
}
```

Output



Conclusion:

Hello World and Basic Calculator programs successfully demonstrate the basic working principles of TypeScript, including the use of typed variables, functions, and the compilation process from TypeScript to JavaScript. They highlight how TypeScript improves code structure, maintainability, and reliability while supporting different execution environments. These examples provide a strong foundation for understanding how TypeScript can be applied to develop simple scripts as well as interactive applications.

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Practical No: 3

Aim: To perform Inheritance example using Typescript.

Theory:
Inheritance in TypeScript is an object-oriented programming concept that allows a class to acquire the properties and methods of another class using the extends keyword
Some Features of Inheritance are:

1. Object-Oriented Concept: Inheritance is a feature of object-oriented programming that allows one class (child class) to acquire the properties and methods of another class (parent class).

2. Parent Class Definition: The parent class Person is used to define common data members such as name and age, along with a method to display these values.

3. Child Class Creation: The child class Student is created using the extends keyword, which establishes an inheritance relationship with the Person class.

4. Constructor Chaining: The super() method is used inside the child class constructor to call the parent class constructor and initialize inherited properties.

5. Method Reusability: The child class can access and use methods of the parent class, such as displayPerson(), which demonstrates code reuse.

6. Additional Properties: The child class introduces its own data members like rollNo and course, which extend the functionality of the parent class.

7. Encapsulation and Structure: The program maintains a clear and structured class design, making the code easier to understand and maintain.

8. Execution Flow: An object of the child class is created, and the child class method is called to display both inherited and child-specific information.

Implementation:

```
// Parent Class
class Person {
  name: string;
  age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  displayPerson(): void {
    console.log("Name: " + this.name);
    console.log("Age: " + this.age);
  }
}

// Child Class (inherits Person)
class Student extends Person {
  rollNo: number;
  course: string;

  constructor(name: string, age: number, rollNo: number, course: string) {
    super(name, age); // Call parent class constructor
    this.rollNo = rollNo;
    this.course = course;
  }

  displayStudent(): void {
    this.displayPerson(); // Access parent method
    console.log("Roll Number: " + this.rollNo);
    console.log("Course: " + this.course);
  }
}
```

```
// Create object of child class
let student1 = new Student("Siddhant", 20, 615, "Information Technology
Engineering");

// Call method
student1.displayStudent();
```

Output:

```
C:\Users\Shilpa_J\Downloads\DatasetFromPDF\FirstTypescriptTutorial>tsc Inheritance.ts

C:\Users\Shilpa_J\Downloads\DatasetFromPDF\FirstTypescriptTutorial>node Inheritance.ts
Name: Siddhant
Age: 20
Roll Number: 615
Course: Information Technology Engineering
```

Conclusion:

The inheritance program successfully demonstrates how TypeScript supports object-oriented principles by enabling a child class to reuse and extend the functionality of a parent class. It shows the use of the extends keyword and super() method to access parent class members and constructors. This improves code maintainability, readability, and scalability, making it easier to build structured and well-organized applications.

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Practical No: 4

Aim: To understand Access Modifiers example using TypeScript.

Theory:

Access Modifiers in Typescript control the visibility and accessibility of class members like properties and methods to outside code. Access Modifiers are a feature of object oriented programming and align with the principles of encapsulation and information hiding.
There are three types of Access Modifiers:

1. Private
    - Class members assigned private are accessible only within the class that they are defined in.
    - If any code or any subclass tries to attempt to access a private member outside the class that private member was declared in will result in an error at compile time.
    - Typescript unlike Ruby allows for cross-instance private access so different instances of the same class can access other instances' private members.
2. Protected
   Class members assigned protected are accessible only within the class that they are defined in and the subclasses that inherit the class members of which that are parent class of.
   If any code tries to attempt to access a protected member outside the class or the subclass that protected member is a part of, will result in an error during compile time.
3. Public

Implementation:

1. Private access specifier implementation

Code:
```
class User{
  private personvalue : string;

  constructor(value:string){
     this.personvalue=value
  }
  public whatIsValue(): string{
```

```
    return this.personvalue
  }
}
const person = new User("Siddhant");
console.log(person.whatIsValue());

//On adding the following line there will be error
console.log(person.personvalue)
//Even derivied classes cannot access private members
//Of the parent class.
class Derived extends User {
 getparentclassvalue() {
   console.log(this.personvalue);
 }
}
```

2. Protected access specifier

Code:

```
class User{
  protected personvalue : string;

  constructor(value:string){
    this.personvalue=value
  }
  public whatIsValue(): string{
    return this.personvalue
  }
}
const person = new User("Siddhant");
console.log(person.whatIsValue());

//On adding the following line there will be error
console.log(person.personvalue)
//Even derivied classes can access protected members
//Of the parent class.
class Derived extends User {
 getparentclassvalue() {
   console.log(this.personvalue);
 }
}
const derivedobj = new Derived("Jadhav");
```

```
console.log(derivedobj.getparentclassvalue());
```

3. Public access specifier

Code:

```
class User{
  public personvalue : string;

  constructor(value:string){
     this.personvalue=value
  }
  public whatIsValue(): string{
     return this.personvalue
  }
}
const person = new User("Siddhant");
console.log(person.whatIsValue());

//On adding the following line there will be error
console.log(person.personvalue)
//Even derivied classes can access protected members
//Of the parent class.
class Derived extends User {
 getparentclassvalue() {
  console.log(this.personvalue);
 }
}
const derivedobj = new Derived("Jadhav");
console.log(derivedobj.getparentclassvalue());
```

Output:

1. For Private access specifier.

```
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$ tsc work.ts
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$ node work.js
Siddhant
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$
```

```
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$ tsc work.ts
work.ts:15:20 - error TS2341: Property 'personvalue' is private and only accessible within class 'User'.

15 console.log(person.personvalue)
                      ~~~~~~~~~~

work.ts:20:22 - error TS2341: Property 'personvalue' is private and only accessible within class 'User'.

20     console.log(this.personvalue);
                       ~~~~~~~~~~


Found 2 errors.

ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$
```

2. For Protected access specifier.

```
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$ tsc work.ts
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$ node work.js
Siddhant
Jadhav
undefined
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$
```

```
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$ tsc work.ts
work.ts:15:20 - error TS2445: Property 'personvalue' is protected and only accessible within class 'User' and its subclasses.

15 console.log(person.personvalue)
                      ~~~~~~~~~~


Found 1 error.
```

3. For Public access specifier.

```
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$ tsc work.ts
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$ node work.js
Siddhant
Siddhant
Jadhav
undefined
ict02@ict02-HP-280-Pro-G6-Microtower-PC:~/Documents$
```

Conclusion:

Access modifiers in TypeScript help control data visibility and enforce encapsulation in object-oriented programming. Private, protected, and public modifiers ensure secure and structured access to class members. By using them effectively, developers can improve code maintainability, prevent unauthorized access, and create scalable, well-organized applications with better data protection.

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Practical No: 5

Aim: A) To write Hello world program using MVC model of AngularJS.
B) Create an application for Students Record using AngularJS.

Theory:

AngularJS is a popular JavaScript framework used to build dynamic web applications. It follows the MVC (Model-View-Controller) architecture, which separates an application into three interconnected components to improve modularity and maintainability.

1. Model: Represents the data and business logic of the application. In AngularJS, the model is managed using JavaScript objects.
2. View: The user interface that displays data to the user. It is created using HTML and AngularJS directives.
3. Controller: Acts as an intermediary between Model and View. It handles user input and updates the model accordingly.

In a Hello World program, the model contains a simple message, the view displays it using expressions like {{message}}, and the controller initializes the message. AngularJS uses two-way data binding, meaning changes in the model automatically update the view.
This approach simplifies development by reducing manual DOM manipulation and ensuring synchronization between data and UI.
A Student Record application using AngularJS demonstrates how MVC architecture can manage structured data efficiently. This application typically stores and displays student details such as name, roll number, marks, and grades.

1. Model: Contains student data stored in arrays or objects.
2. View: Displays the student list using AngularJS directives like ng-repeat.
3. Controller: Manages operations such as adding, updating, or deleting student records.

AngularJS features like data binding, filters, and directives help in dynamically updating the UI when data changes. For example, when a new student is added, the view updates automatically without reloading the page.
This application improves understanding of real-time data handling, form validation, and CRUD operations (Create, Read, Update, Delete) in AngularJS.

Code:
A) To write Hello world program using MVC model of AngularJS.

```
<!DOCTYPE html>
<html ng-app="myApp">
```

```
<head>
   <title>Hello World MVC</title>
   <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body>

<div ng-controller="myController">
   <h1>{{message}}</h1>
</div>

<script>
   var app = angular.module("myApp", []);

   app.controller("myController", function($scope){
      $scope.message = "Hello World using AngularJS MVC!";
   });
</script>

</body>
</html>
```

B) Create an application for Students Record using AngularJS.

```
<!DOCTYPE html>
<html ng-app="studentApp">
<head>
   <title>Student Record</title>
   <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
   <style>
      table, th, td {
         border: 1px solid black;
         border-collapse: collapse;
         padding: 8px;
      }
   </style>
</head>
<body>

<div ng-controller="studentController">
```

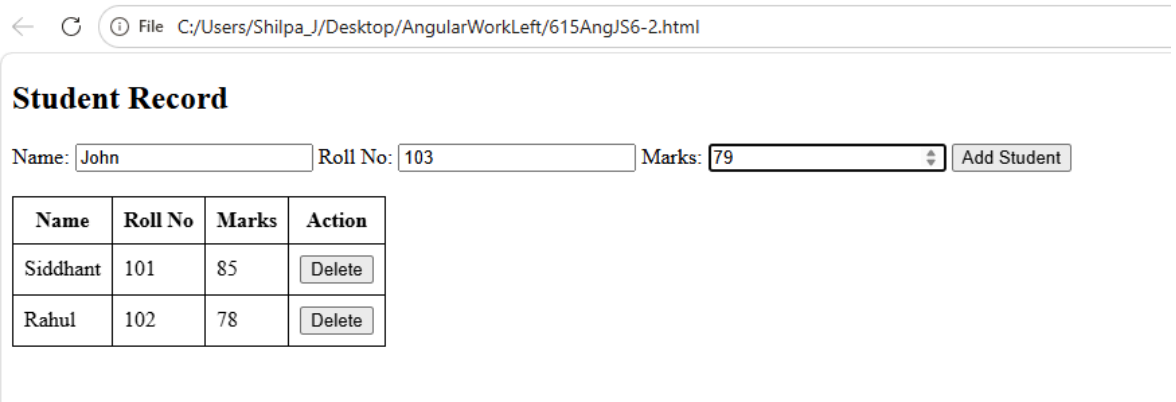
```
    <h2>Student Record</h2>

    <!-- Form to Add Student -->
    Name: <input type="text" ng-model="newStudent.name">
    Roll No: <input type="text" ng-model="newStudent.roll">
    Marks: <input type="number" ng-model="newStudent.marks">
    <button ng-click="addStudent()">Add Student</button>

    <br><br>

    <!-- Display Student List -->
    <table>
        <tr>
            <th>Name</th>
            <th>Roll No</th>
            <th>Marks</th>
            <th>Action</th>
        </tr>
        <tr ng-repeat="student in students">
            <td>{{student.name}}</td>
            <td>{{student.roll}}</td>
            <td>{{student.marks}}</td>
            <td>
                <button ng-click="deleteStudent($index)">Delete</button>
            </td>
        </tr>
    </table>

</div>

<script>
    var app = angular.module("studentApp", []);

    app.controller("studentController", function($scope){

        // Initial student data
        $scope.students = [
            {name: "Siddhant", roll: "101", marks: 85},
            {name: "Rahul", roll: "102", marks: 78}
        ];
```

```
    // Add new student
    $scope.addStudent = function(){
        $scope.students.push($scope.newStudent);
        $scope.newStudent = {};
    };

    // Delete student
    $scope.deleteStudent = function(index){
        $scope.students.splice(index, 1);
    };

  });
</script>

</body>
</html>
```

Implementation:

A) To write Hello world program using MVC model of AngularJS.

B) Create an application for Students Record using AngularJS.



Student Record

Name: Roll No: Marks: Add Student

Name	Roll No	Marks	Action
Siddhant	101	85	Delete
Rahul	102	78	Delete
John	103	79	Delete

Conclusion:
AngularJS simplifies web application development through its MVC architecture and two-way data binding. Creating Hello World and Student Record applications helps understand core concepts like controllers, models, and views. These applications demonstrate efficient data handling, improved code organization, and dynamic user interface updates, making AngularJS a powerful tool for beginners.

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Practical No: 6

Aim: A) Write a Basic JavaScript program using Ajax
B) Write a program to use AJAX for user validation using and to show the result on the same page below the submit button

Theory:
AJAX (Asynchronous JavaScript and XML) is a technique used in web development to send and receive data from a server asynchronously without reloading the entire web page. It improves user experience by making web applications faster and more interactive.
In AJAX, JavaScript communicates with the server using objects like XMLHttpRequest or modern APIs such as Fetch API. The request is sent in the background, and when the server responds, only a specific part of the webpage is updated dynamically.
Key features of AJAX:

1. Asynchronous communication with the server
2. No full page reload required
3. Faster and smoother user experience
4. Data can be exchanged in formats like JSON, XML, or text

Working of AJAX:

1. A user triggers an event (click, submit, etc.)
2. JavaScript creates an AJAX request
3. The request is sent to the server
4. The server processes the request and sends a response
5. JavaScript updates the webpage dynamically

AJAX is widely used in modern applications like live search, auto-fill forms, chat applications, and dynamic content loading.

User validation using AJAX is a technique where user input (such as username, email, or password) is checked with the server without reloading the page. This provides immediate feedback to the user and enhances usability.
In this method, when the user submits a form or enters data, an AJAX request is sent to the server containing the input values. The server verifies the data (e.g., checking if a username already exists) and returns a response. This response is then displayed on the same webpage below the submit button.
Advantages of AJAX validation:

1. Real-time validation without page refresh
2. Better user experience
3. Reduced server load (partial requests instead of full page reloads)
4. Faster response and interaction

Working process:

1. User enters data in a form
2. JavaScript captures the input and sends it via AJAX
3. Server processes the validation logic
4. Server returns success/error message
5. JavaScript displays the result dynamically on the same page

This technique is commonly used in login systems, registration forms, and email/username availability checks.

Code:

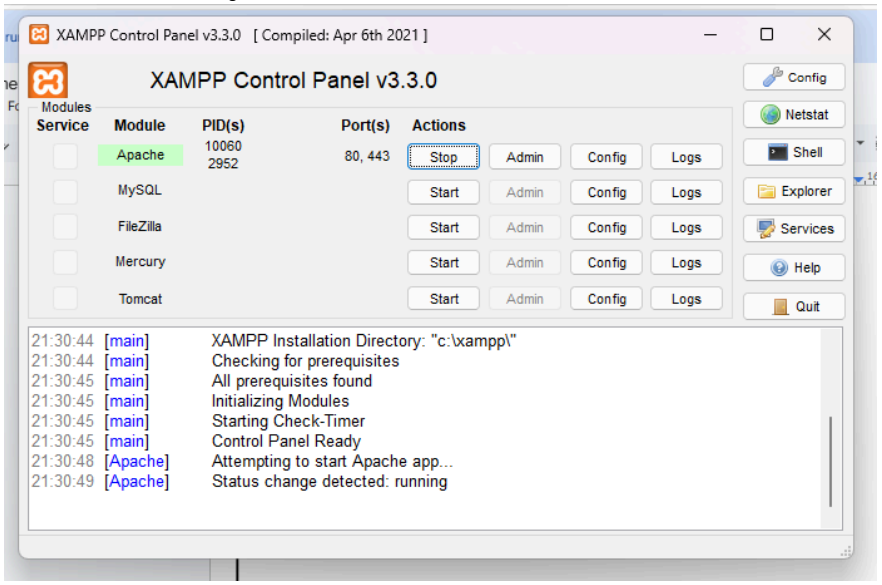
```
<!DOCTYPE html>
<html>
<head>
   <title>AJAX User Validation</title>
</head>
<body>

<h2>User Validation Form</h2>

<form onsubmit="validateUser(event)">
   Username: <input type="text" id="username" required>
   <br><br>
   <button type="submit">Submit</button>
</form>

<p id="message"></p>

<script>
function validateUser(event) {
   event.preventDefault();

   var username = document.getElementById("username").value;
   var xhr = new XMLHttpRequest();

   xhr.onreadystatechange = function() {
      if (xhr.readyState == 4 && xhr.status == 200) {
         document.getElementById("message").innerHTML = xhr.responseText;
      }
   };

   xhr.open("GET", "validate.php?user=" + username, true);
   xhr.send();
```

```
}
</script>

</body>
</html>

<?php
$valid_users = array("admin", "user123", "test");

$username = $_GET['user'];

if (in_array($username, $valid_users)) {
   echo "Username is valid!";
} else {
   echo "Username not found!";
}
?>
```

Implementation:

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**



Conclusion:
AJAX-based user validation improves web applications by providing real-time feedback without reloading the page. It enhances user experience, reduces server load, and increases efficiency. By validating inputs instantly, it minimizes errors and speeds up interaction, making applications more dynamic, responsive, and user-friendly in modern web development environments.

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Practical No: 7

Aim: Design and develop small web application using AJAX, HTML and JSP.

Theory:

AJAX is a web development technique used to create fast, dynamic, and interactive web applications. It allows data to be exchanged between the client and server asynchronously without reloading the entire web page. AJAX uses JavaScript along with objects like XMLHttpRequest or Fetch API to send requests and receive responses in the background. This improves the performance and responsiveness of web applications by updating only specific parts of a webpage instead of refreshing the whole page.

Features of AJAX:
1. Asynchronous communication with the server
2. No full page reload required
3. Faster user interaction and improved performance
4. Supports multiple data formats like JSON, XML, and text
5. Enhances user experience with dynamic updates

Advantages of AJAX:
1. Reduces server load and bandwidth usage
2. Provides real-time updates
3. Makes web applications more interactive
4. Improves speed and efficiency

JSP (Java Server Pages)

JSP is a server-side technology used to create dynamic web content. It allows developers to embed Java code within HTML pages to process client requests and generate responses dynamically. JSP runs on a web server such as Apache Tomcat and is automatically converted into a servlet before execution. It is widely used for building web applications that require database interaction, form handling, and business logic processing.

Features of JSP:
1. Supports embedding Java code in HTML
2. Platform-independent and portable
3. Automatically converted into servlets
4. Supports reusable components like tags and beans
5. Easy integration with Java frameworks

Advantages of JSP:

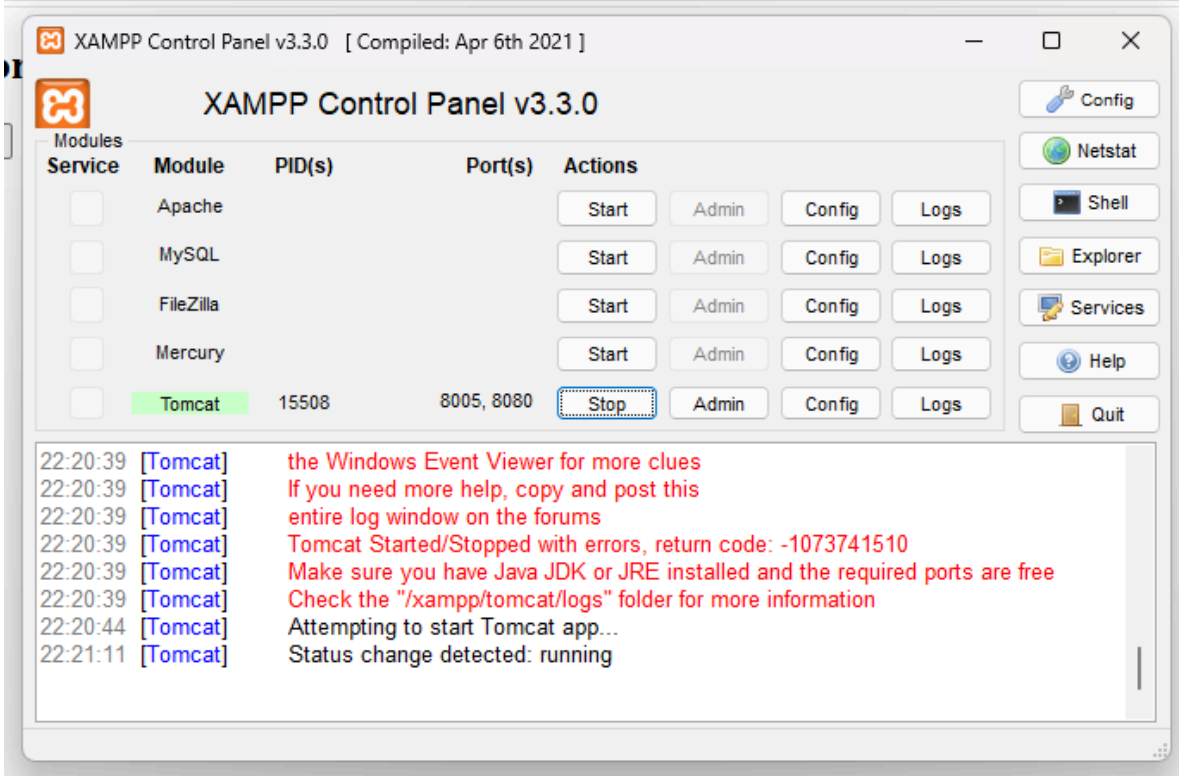
1. Simplifies dynamic content creation
2. Separates presentation and business logic
3. Reusable and maintainable code
4. Efficient and scalable for large applications

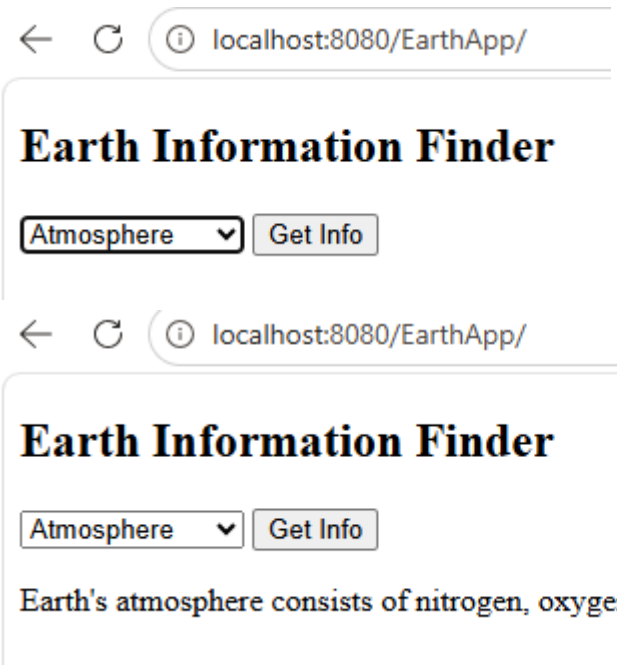
Code:

```
<%
String topic = request.getParameter("topic");

if(topic == null) {
   out.println("No topic selected");
}
else if(topic.equals("atmosphere")) {
   out.println("Earth's atmosphere consists of nitrogen, oxygen, and other gases that
support life.");
}
else if(topic.equals("layers")) {
   out.println("Earth has four main layers: crust, mantle, outer core, and inner core.");
}
else if(topic.equals("population")) {
   out.println("Earth has a population of over 8 billion people.");
}
else {
   out.println("Invalid topic");
}
%>
<!DOCTYPE html>
<html>
<head>
   <title>Earth Information App</title>
</head>
<body>

<h2>Earth Information Finder</h2>

<select id="topic">
   <option value="">--Select Topic--</option>
   <option value="atmosphere">Atmosphere</option>
   <option value="layers">Layers</option>
   <option value="population">Population</option>
</select>
```

```
<button onclick="getInfo()">Get Info</button>

<p id="result"></p>

<script>
function getInfo() {
   var topic = document.getElementById("topic").value;

   if (topic == "") {
      alert("Please select a topic");
      return;
   }

   var xhr = new XMLHttpRequest();

   xhr.onreadystatechange = function() {
      if (xhr.readyState == 4 && xhr.status == 200) {
         document.getElementById("result").innerHTML = xhr.responseText;
      }
   };

   xhr.open("GET", "earth.jsp?topic=" + topic, true);
   xhr.send();
}
</script>

</body>
</html>
```

Implementation:


XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]
XAMPP Control Panel v3.3.0
Modules
Service Module PID(s) Port(s) Actions
Apache Start Admin Config Logs
MySQL Start Admin Config Logs
FileZilla Start Admin Config Logs
Mercury Start Admin Config Logs
Tomcat 15508 8005, 8080 Stop Admin Config Logs
Config
Netstat
Shell
Explorer
Services
Help
Quit
22:20:39 [Tomcat] the Windows Event Viewer for more clues
22:20:39 [Tomcat] If you need more help, copy and post this
22:20:39 [Tomcat] entire log window on the forums
22:20:39 [Tomcat] Tomcat Started/Stopped with errors, return code: -1073741510
22:20:39 [Tomcat] Make sure you have Java JDK or JRE installed and the required ports are free
22:20:39 [Tomcat] Check the "/xampp/tomcat/logs" folder for more information
22:20:44 [Tomcat] Attempting to start Tomcat app...
22:21:11 [Tomcat] Status change detected: running


localhost:8080/EarthApp/
Earth Information Finder
Atmosphere
Get Info
localhost:8080/EarthApp/
Earth Information Finder
Atmosphere
Get Info
Earth's atmosphere consists of nitrogen, oxygen, and other gases that support life.

Conclusion:
AJAX and JSP together help in developing modern, efficient, and user-friendly web applications. AJAX handles client-side asynchronous communication, while JSP manages server-side processing. This combination enables faster data exchange, reduces page reloads, and enhances the overall performance and user experience of web applications.

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Practical No: 8

Aim: To Build a RESTful API using MongoDB. Also using HTTP Request Method(GET,POST,PUT, DELETE)

Theory:
A RESTful API (Representational State Transfer) is a web service architecture that enables communication between a client and a server using standard HTTP protocols. It is stateless, meaning each request from the client contains all the information needed for the server to process it. RESTful APIs are widely used for developing scalable and efficient web applications.
MongoDB is a NoSQL database that stores data in a flexible, JSON-like format called BSON (Binary JSON). It is schema-less, allowing developers to store and manage data easily without a fixed structure. MongoDB is commonly used with RESTful APIs to perform CRUD operations.
In a RESTful API, HTTP request methods are used to interact with resources:
HTTP Methods:

1. GET:

Used to retrieve data from the server. It does not modify data.
Example: Fetching user details from the database.

2. POST:

Used to send data to the server to create a new resource.
Example: Adding a new user to MongoDB.

3. PUT:

Used to update an existing resource on the server.
Example: Updating user information.

4. DELETE:

Used to remove a resource from the server.
Example: Deleting a user record from MongoDB.
Working of RESTful API with MongoDB:

1. The client sends an HTTP request (GET, POST, PUT, DELETE).
2. The server (API) receives the request and processes it.
3. The API interacts with MongoDB to perform the required operation.
4. MongoDB returns the result to the server.
5. The server sends a response back to the client in JSON format.

Advantages:

1. Scalable and flexible architecture
2. Efficient data handling using JSON
3. Easy integration with web and mobile applications
4. Supports fast development and performance

Code:

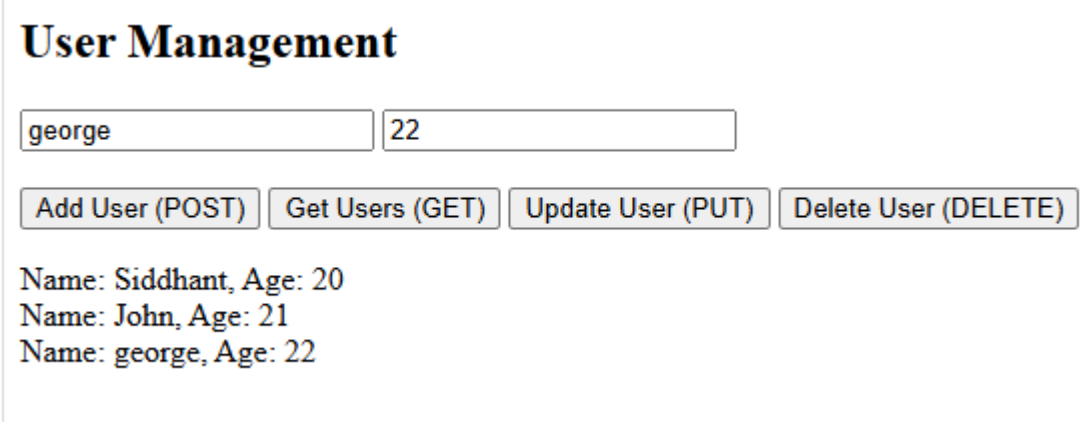
```
<!DOCTYPE html>
<html>
<head>
   <title>Express MongoDB API Test</title>
</head>
<body>

<h2>User Management</h2>

<input type="text" id="name" placeholder="Enter Name">
<input type="number" id="age" placeholder="Enter Age">

<br><br>

<button onclick="addUser()">Add User (POST)</button>
<button onclick="getUsers()">Get Users (GET)</button>
<button onclick="updateUser()">Update User (PUT)</button>
<button onclick="deleteUser()">Delete User (DELETE)</button>

<p id="result"></p>

<script>

//  POST
function addUser() {
   let name = document.getElementById("name").value;
   let age = document.getElementById("age").value;

   fetch("http://localhost:3000/users", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({ name, age })
   })
```

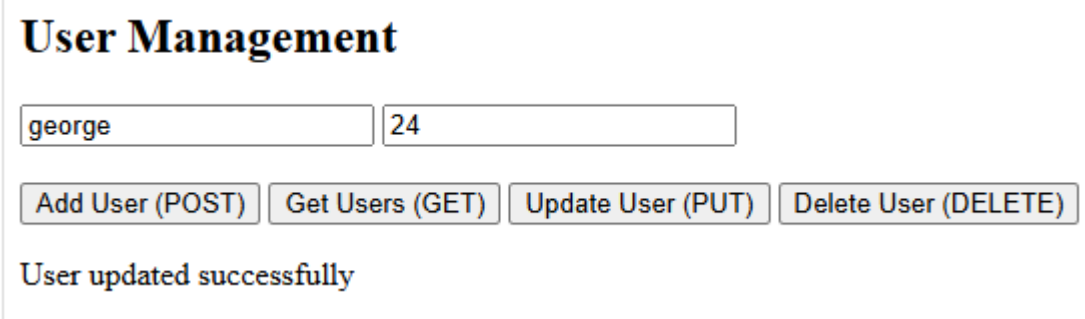
```
    .then(res => res.json())
    .then(data => document.getElementById("result").innerHTML = data.message);
}

// GET
function getUsers() {
    fetch("http://localhost:3000/users")
    .then(res => res.json())
    .then(data => {
        let output = "";
        data.forEach(user => {
            output += `Name: ${user.name}, Age: ${user.age} <br>`;
        });
        document.getElementById("result").innerHTML = output;
    });
}

// PUT
function updateUser() {
    let name = document.getElementById("name").value;
    let age = document.getElementById("age").value;

    fetch("http://localhost:3000/users/" + name, {
        method: "PUT",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({ age })
    })
    .then(res => res.json())
    .then(data => document.getElementById("result").innerHTML = data.message);
}

// DELETE
function deleteUser() {
    let name = document.getElementById("name").value;

    fetch("http://localhost:3000/users/" + name, {
        method: "DELETE"
    })
    .then(res => res.json())
```

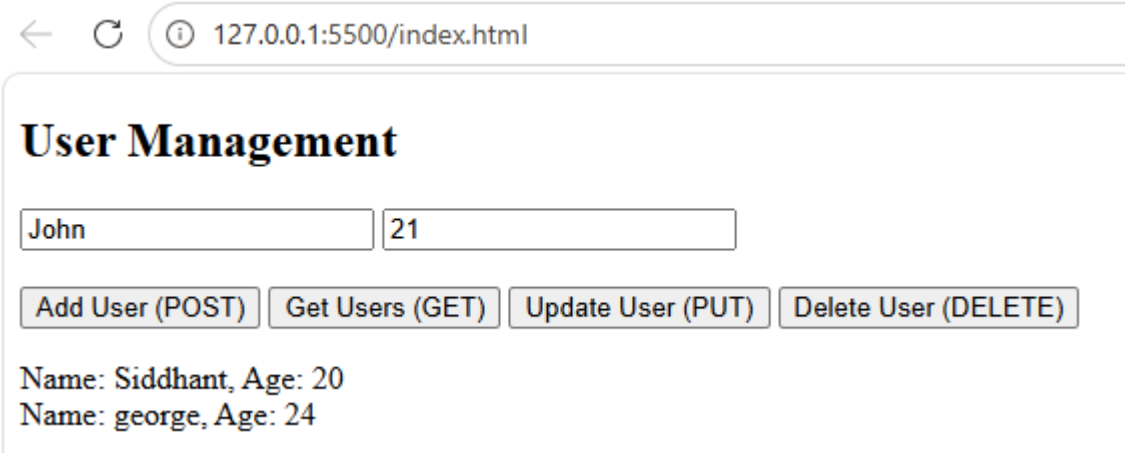
```
    .then(data => document.getElementById("result").innerHTML = data.message);
}

</script>

</body>
</html>
const express = require('express');
const { MongoClient } = require('mongodb');
const cors = require('cors');

const app = express();
app.use(express.json());
app.use(cors());

// MongoDB Connection URL
const url = "mongodb://127.0.0.1:27017";
const client = new MongoClient(url);

let db;

// Connect to MongoDB
async function connectDB() {
    await client.connect();
    db = client.db("mydatabase");
    console.log("MongoDB Connected");
}
connectDB();

// GET (Read All Users)
app.get('/users', async (req, res) => {
    const users = await db.collection('users').find().toArray();
    res.json(users);
});

// GET (Single User)
app.get('/users/:name', async (req, res) => {
    const user = await db.collection('users').findOne({ name: req.params.name });
    res.json(user);
```

```
});

// POST (Create User)
app.post('/users', async (req, res) => {
    const newUser = req.body;

    await db.collection('users').insertOne(newUser);

    res.json({ message: "User added successfully" });
});

// PUT (Update User)
app.put('/users/:name', async (req, res) => {
    const name = req.params.name;
    const updatedData = req.body;

    await db.collection('users').updateOne(
        { name: name },
        { $set: updatedData }
    );

    res.json({ message: "User updated successfully" });
});

// DELETE (Delete User)
app.delete('/users/:name', async (req, res) => {
    const name = req.params.name;

    await db.collection('users').deleteOne({ name: name });

    res.json({ message: "User deleted successfully" });
});


// Start Server
app.listen(3000, () => {
    console.log("Server running on http://localhost:3000");
});
```

Implementation:







Conclusion:
Building a RESTful API using MongoDB and HTTP methods enables efficient data management and communication between client and server. It simplifies CRUD operations and helps in developing modern, scalable, and high-performance web applications.

MANJARA CHARITABLE TRUST

RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI

Permanently Affiliated to University of Mumbai

Department of Information Technology

NBA Accredited

A.Y. 2025-2026

Class: TE-IT **Semester: VI**

Subject:Web Lab **Roll no: 615**

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Practical No: 9

Aim: To implement Design Feedback Form using Flask.

Theory:

A feedback form using Flask is a web application that allows users to submit their opinions, suggestions, or comments through a web interface. Flask is a lightweight Python web framework that enables easy handling of client-server communication and dynamic content generation. In this application, the frontend is created using HTML and CSS, where users enter details such as name, email, and feedback message.When the form is submitted, the data is sent to the Flask server using the HTTP POST method.
Flask processes the request through defined routes and retrieves the input using request objects. The backend can validate the data and optionally store it in a database or file. After processing, a response such as a success or thank-you message is sent back to the user. This application demonstrates how Flask connects the frontend and backend to build interactive web applications.

Key Components

1. Frontend (HTML/CSS)
   Used to design the feedback form interface
2. Flask Backend
   Handles routing and processes user input
3. HTTP Methods (POST)
   Used to send form data to the server
4. Request Handling
   Flask retrieves user data using request objects

Features

1. Simple and user-friendly interface
2. Collects user feedback efficiently
3. Dynamic response without page reload (if AJAX used)
4. Easy to extend with database storage

Advantages

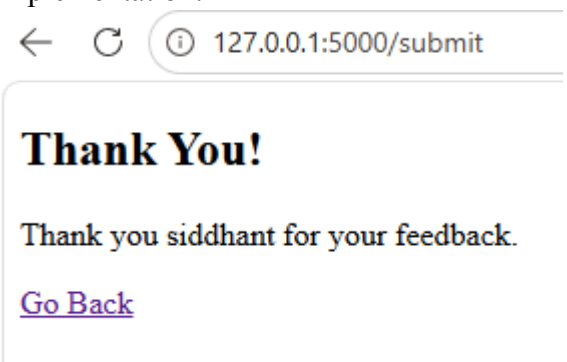
1. Lightweight and easy to implement
2. Flexible and scalable
3. Supports rapid development
4. Useful for real-world applications like surveys and reviews

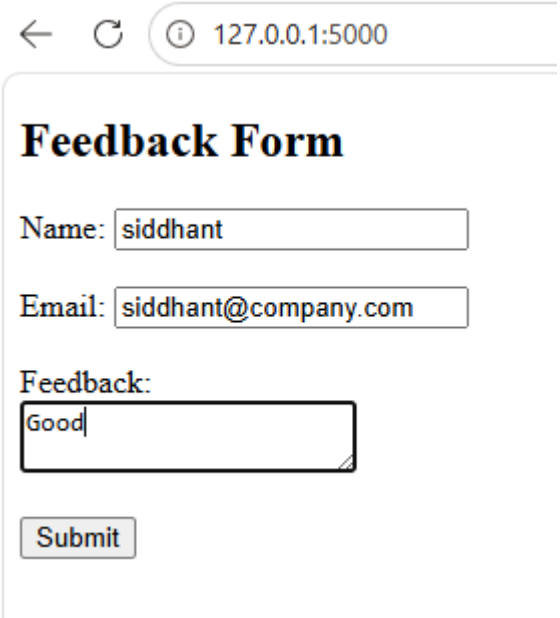
Code:

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('form.html')

@app.route('/submit', methods=['POST'])
def submit():
    name = request.form['name']
    email = request.form['email']
    feedback = request.form['feedback']
    return render_template('result.html', name=name)

if __name__ == '__main__':
    app.run(debug=True)

<!DOCTYPE html>
<html>
<head>
    <title>Thank You</title>
</head>
<body>

<h2>Thank You!</h2>
<p>Thank you {{name}} for your feedback.</p>

<a href="/">Go Back</a>

</body>
</html>

<!DOCTYPE html>
<html>
<head>
    <title>Feedback Form</title>
</head>
```

```
<body>

<h2>Feedback Form</h2>

<form action="/submit" method="POST">
   Name: <input type="text" name="name" required><br><br>
   Email: <input type="email" name="email" required><br><br>
   Feedback:<br>
   <textarea name="feedback" required></textarea><br><br>

   <button type="submit">Submit</button>
</form>

</body>
</html>
```

Implementation:





Conclusion:

The feedback form designed using Flask provides an efficient method for collecting user input and processing it dynamically. It demonstrates the integration of frontend and backend technologies, enabling smooth data handling. This approach is simple, flexible, and widely used for developing interactive and responsive web applications.

A.Y. 2025-2026

Class: TE-IT **Semester: VI**
Subject:Web Lab **Roll no: 615**

Practical No: 10

Aim: Design Weather App using Flask

Theory:

A Weather Application using Flask is a web-based application that demonstrates how Flask can be used to build dynamic and interactive web applications. Flask is a lightweight Python web framework that provides essential tools and libraries to handle routing, request processing, and response generation. It follows a simple architecture and allows developers to quickly develop web applications with minimal configuration.
In this application, Flask acts as the backend server that handles user requests and communicates with external services such as a weather API. The frontend, developed using HTML and CSS, allows users to enter a city name. When the form is submitted, Flask receives the request using HTTP methods, processes it, and fetches relevant weather data from an external API. The processed data is then rendered dynamically using templates and displayed to the user.
Flask uses built-in features like routing (@app.route), template rendering (render_template), and request handling (request) to manage the flow of the application. This demonstrates how Flask integrates frontend and backend seamlessly to create efficient web applications.
Flask Concepts Used

1. Routing Defines URLs and maps them to Python functions
2. Templates (Jinja2):Used to dynamically display data in HTML pages
3. Request Handling Retrieves user input using request.form or request.args
4. Response Handling: Sends processed data back to the client
5. Integration with APIs: Fetches external data (weather information)

Role of Weather Application

1. Accepts user input (city name)

Sends request to Flask server

2. Flask fetches data from weather API

Displays temperature, humidity, and conditions
Features

1. Dynamic web application using Flask
2. Real-time data retrieval
3. Simple and user-friendly interface
4. Efficient client-server communication

Advantages of Using Flask

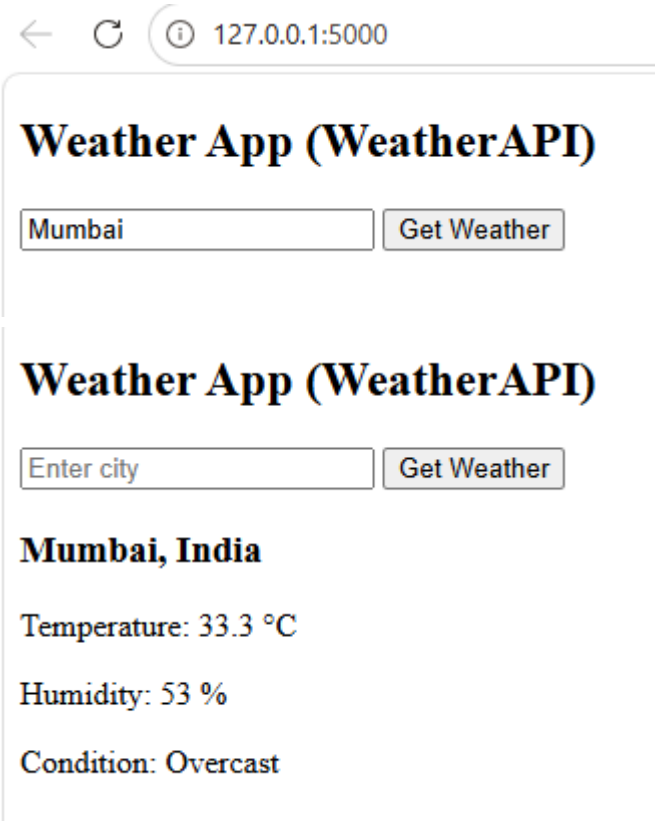
1. Lightweight and easy to learn
2. Flexible and scalable
3. Minimal setup required
4. Supports rapid development
5. Easy integration with APIs and databases

Code:

```
from flask import Flask, render_template, request
import requests

app = Flask(__name__)

API_KEY = "3742c1c82d58478481e84607261604"

@app.route('/', methods=['GET', 'POST'])
def index():
    weather = None

    if request.method == 'POST':
        city = request.form['city']

        url = f"http://api.weatherapi.com/v1/current.json?key={API_KEY}&q={city}"

        response = requests.get(url)
        data = response.json()

        if "error" not in data:
            weather = {
                "city": data["location"]["name"],
                "country": data["location"]["country"],
                "temperature": data["current"]["temp_c"],
                "humidity": data["current"]["humidity"],
                "condition": data["current"]["condition"]["text"]
            }
        else:
            weather = {"error": "City not found"}

    return render_template('index.html', weather=weather)
```

```
if __name__ == '__main__':
    app.run(debug=True)
<!DOCTYPE html>
<html>
<head>
    <title>Weather App</title>
</head>
<body>

<h2>Weather App (WeatherAPI)</h2>

<form method="POST">
    <input type="text" name="city" placeholder="Enter city" required>
    <button type="submit">Get Weather</button>
</form>

{% if weather %}
    {% if weather.error %}
        <p>{{ weather.error }}</p>
    {% else %}
        <h3>{{ weather.city }}, {{ weather.country }}</h3>
        <p>Temperature: {{ weather.temperature }} °C</p>
        <p>Humidity: {{ weather.humidity }} %</p>
        <p>Condition: {{ weather.condition }}</p>
    {% endif %}
{% endif %}

</body>
</html>
```

Implementation:



Conclusion:
The Weather App developed using Flask demonstrates how web applications can integrate external APIs to provide real-time data. It highlights the interaction between frontend and backend components and shows how Flask can be used to build simple, efficient, and dynamic web applications for practical use.